You can now build a new `SecureTokenStore` implementation using macaroons as shown in listing 9.15. To create a macaroon, you'll first use another `TokenStore` implementation to generate a unique ID. You can use any of the existing stores, but for simplicity you'll use the `JsonTokenStore` in these examples (see chapter 5). The JSON is protected against tampering by the HMAC authentication tag.

> **TIP** The `JsonTokenStore` returns a base64-encoded string, which will then be base64-encoded again by the macaroon library. A more efficient implementation would use the unencoded JSON as the identifier.

You then create the macaroon using the `MacaroonsBuilder.create()` method, passing in the identifier and the HMAC key. You can also give an optional hint for where the macaroon is intended to be used. Because you'll be using these with capability URIs that already include the full location, you can leave that field blank to save space. You can then use the `macaroon.serialize()` method to convert the macaroon into a URL-safe base64 string format. To verify a macaroon, you deserialize and validate the macaroon, which will verify the HMAC tag and check any caveats. If the macaroon is valid then you can look up the identifier in the delegate token store.

To revoke a macaroon, you simply deserialize and revoke the identifier. For simplicity you'll skip verifying the caveats for revocation because the `JsonTokenStore` doesn't support it, but you may want to in a production implementation if malicious token revocation might be a concern. In the same Natter API project you've been using so far, navigate to